

Token economics for multi-phase autonomous agent systems

Prompt caching is the single highest-leverage optimization for agent workloads making 200+ LLM calls per job—delivering 20–40% input cost reduction with minimal implementation effort. Anthropic's explicit caching offers a **90% read discount** (claude) (Claude API Docs) with deterministic hits, (Claude API Docs) dwarfing OpenAI's automatic 50% discount (OpenAI) with probabilistic routing. (OpenRouter) The optimal prompt ordering—static content first, dynamic content last—can push 20–30% of every call's tokens into the cached prefix. Combined with tool schema compression, workspace budgets, and tiered context compaction, a well-architected system can cut total input costs by **40–55%**, turning a \$17 job into a \$9 one on Claude Sonnet. (AI Free API)

The key architectural insight across all six research areas is that **conversation history dominates costs**, not the fixed components the user is considering optimizing. Tool schemas at 5,000 tokens are eclipsed by average conversation histories of 20,000+ tokens. Every strategy should therefore prioritize keeping conversation history compact while caching the stable prefix.

Prompt caching saves 20–40% but varies wildly by provider

All three major backends support prefix caching, but their economics differ dramatically.

Anthropic's explicit caching is the most powerful for agent workloads. Cache reads cost **\$0.30/MTok**—a 90% discount from the \$3.00/MTok base for Sonnet 4. Cache writes incur a 25% surcharge (\$3.75/MTok for a 5-minute TTL) (Claude API Docs) (claude) or 100% surcharge (\$6.00/MTok for 1-hour TTL). (Claude API Docs) (Claude API Docs) The minimum cacheable prefix is **1,024 tokens** for Sonnet/Opus 4.x models, (AWS) with up to **4 explicit breakpoints** per request. (Prompt Hub) (claude) The cache processes in a fixed hierarchy—tools → system → messages—meaning any change to tool definitions invalidates everything downstream. (Claude API Docs +2) The TTL auto-refreshes on every hit, (Google) so sequential agent calls within 5 minutes maintain the cache indefinitely.

OpenAI's automatic caching requires zero code changes but delivers weaker results. (OpenRouter) The discount is **50% for GPT-4o** (\$1.25/MTok cached vs \$2.50 base) (Price Per Token) and **90% for GPT-5** (\$0.125/MTok vs \$1.25 base). There's no write surcharge. (OpenAI) However, routing is hash-based and probabilistic— independent testing shows only **~50% cache hit rate** even for identical prefixes, because requests aren't guaranteed to reach the same server. (ngrok) The minimum prefix is also 1,024 tokens, (LiteLLM) increasing in 128-token increments. (OpenAI)

vLLM prefix caching operates at the compute layer rather than billing. It reuses KV-cache blocks (vLLM) (default **16 tokens per block** on CUDA) via hash-chain matching built on PagedAttention. (vLLM) There's no TTL—blocks persist as long as GPU memory allows. For self-hosted models, savings manifest as reduced prefill latency (up to 85% TTFT reduction for long cached prefixes) rather than per-token cost reductions. (Claude) (vLLM) Every complete 16-token block in the stable prefix gets reused.

For the user's workload—200 sequential calls with a stable 7,600-token prefix (tools + system prompt + instructions.md)—Anthropic caching saves **~\$4.00 per job** on input costs alone. OpenAI saves roughly **~\$0.95** due to the lower discount and probabilistic hits. The savings percentage shrinks as conversation history grows:

from ~66% early in a job (when history is ~2,000 tokens) to ~8% late in a job (when history reaches 80,000 tokens). The absolute savings per call remain constant regardless.

Ordering prompts by stability is the foundational optimization

Because all three providers use strict prefix matching, the optimal structure places components in descending order of stability:

Position	Component	Tokens	Change frequency	Cache behavior
1st (tools param)	Tool schemas	3,000–5,000	Rarely	Cached on nearly every call
2nd (system)	System prompt	600	Never	Cached (part of prefix)
3rd (system)	instructions.md	2,000	Until compaction (~every 50 calls)	Cached within compaction window
4th (system)	Phase prompt	300	Every 10–50 calls	Cached within phase
5th (system)	workspace.md	300–800	Most calls	Rarely cached
6th (messages)	Conversation history	2,000–80,000	Every call	Never cached from prefix

For Anthropic, place **cache breakpoints** after tool schemas (BP1), after instructions.md (BP2), after the phase prompt (BP3), and at the end of conversation history (BP4). The 20-block lookback window means you need explicit breakpoints if your messages array exceeds ~20 content blocks. [Claude API Docs](#) [claude](#) A critical detail: Anthropic's system prompt minimum of **1,024 tokens** [AWS](#) means the 600-token system prompt alone won't cache—it must be combined with tool schemas (which come first in the hierarchy) to cross the threshold.

For OpenAI, **tool definition ordering must be deterministic** across calls. If your framework serializes JSON with non-deterministic key ordering (common in Go or Swift), this silently breaks caching. [claude](#) Use sorted keys or a fixed schema order. The [prompt_cache_key](#) parameter can improve routing consistency for recurring prefixes. [OpenAI](#)

For vLLM, the 16-token block granularity means $\lfloor 7,600/16 \rfloor = 475$ blocks of stable prefix get reused. Partial blocks at the boundary are discarded. Ensure your prompt template concatenates static content without variable-length padding that shifts block boundaries.

Tool schemas deserve compression and dynamic loading

With 20–40 tools consuming 3,000–8,000 tokens, tool definitions represent 5–10% of every call. [Tool2agent](#) Each provider handles them differently, and the optimization strategies vary accordingly.

Anthropic injects a hidden system prompt wrapping tool definitions, adding per-model overhead. (AWS) Tools sit first in the cache hierarchy, (claude) making them ideal candidates for caching—after the first call, 5,000 tokens of tool schemas cost only **500 effective tokens** ($0.1\times$). Additionally, Claude 4 models include built-in **token-efficient tool use**, reducing output token consumption on tool calls by **14–70%** without any code changes. (Claude)

OpenAI internally converts JSON Schema to TypeScript type declarations, adding formatting overhead. (hmarr) A function with detailed descriptions can consume more tokens than the raw JSON suggests due to this internal transformation. The tools parameter participates in automatic caching, but the serialized form must be byte-identical across calls.

For open-source models via vLLM, tool schemas are injected via Jinja2 chat templates directly into the prompt text. This gives you complete control—shorter descriptions translate directly to fewer tokens with no hidden overhead.

The most impactful optimization is **Anthropic's Tool Search Tool pattern**, which achieved **85% reduction** in tool definition tokens in production. (Anthropic) Instead of loading all tools, you send a single search tool (~500 tokens) and load relevant tools on demand. Anthropic's internal system went from **134K tokens** of tool definitions to a fraction of that. For 58 MCP tools consuming ~55K tokens, Tool Search reduced this to ~8.7K. Accuracy actually *improved*: Opus 4 went from 49% → 74% because the model saw fewer irrelevant tools. (Anthropic)

For users who don't want the complexity of dynamic loading, **description compression** offers 30–50% savings per tool with minimal accuracy impact. (OpenAI) The key insight from research is that **tool names carry the most semantic weight** for selection. A tool named (web_search) with description "Search web" performs nearly identically to one with a paragraph-long explanation, provided the parameter names are also self-explanatory. Safe to remove: optional parameter descriptions, default values, pattern fields. Unsafe to remove: tool names, (required) arrays, type information, and descriptions that disambiguate between similar tools.

Combining caching with compression reduces effective tool token cost from 5,000 tokens at full price to roughly **300–500 effective tokens per call** (compressed schemas cached at $0.1\times$). (Claude API Docs)

On-demand reference retrieval saves tokens but caching changes the math

The raw arithmetic for moving 1,500 tokens of reference material from always-in-context to read-on-demand is compelling: **300,000 tokens** ($1,500 \times 200$ calls) versus **~9,000 tokens** ($5 \text{ reads} \times 1,500 + \text{tool overhead}$). That's a 97% reduction on that component.

However, **prompt caching fundamentally alters this tradeoff**. If the 1,500 tokens sit in the cached prefix (after the system prompt but before dynamic content), they cost only $0.1\times$ the base rate on cache hits. The effective cost becomes: 1 cache write at $1.25\times$ (1,875 effective tokens) + 199 cache reads at $0.1\times$ (29,850 effective tokens) = **~31,725 effective tokens**. On-demand retrieval at ~9,000 tokens still wins, but the margin shrinks from 291,000 tokens to **~22,725 tokens**—a saving of just **\$0.07 per job** on Claude Sonnet.

The reliability risk is real. Research on agent failure taxonomies identifies "**Memory Retrieval Failure**" as a core failure mode—agents forgetting to fetch reference material they need. (alphaXiv) The agent may confidently proceed without the reference, hallucinating specifications instead of retrieving them. This is especially dangerous for rule-type or specification-type references where wrong assumptions cascade.

The recommendation is clear: with prompt caching enabled, **keep critical reference material in the cached prefix**. The marginal cost is negligible (~\$0.07/job), and the reliability benefit far outweighs it. Move reference material to on-demand only if it's rarely needed (accessed in <5% of calls) or if it's supplementary rather than normative.

Tiered context compaction outperforms any single strategy

The user's current approach—keeping the 10 most recent tool results—is a basic selective retention strategy. Research shows that a **tiered/cascading approach** significantly outperforms any single compaction method across both token savings and coherence maintenance.

Tier 1 — Observation masking (at 50% context capacity): Hide tool output contents from older turns while preserving all reasoning, actions, and tool invocation records. JetBrains/TUM research found this **matched LLM summarization** in both cost savings and problem-solving ability—with zero extra LLM calls. (jetbrains) The "middle-out" variant (used by Cline and Codex) preserves the first few messages (original goal) and last 10 turns verbatim, masking observations in between. Token savings: **50–80%** of observation content.

Tier 2 — Structured summarization (at 70% capacity): When masking alone isn't sufficient, generate a structured summary with explicit sections: session intent, key decisions, file modifications, current state, errors encountered, and next steps. Factory.ai's evaluation found structured summarization scored **3.70/5** overall versus Anthropic's 3.44 and OpenAI's 3.35 on unstructured approaches. (factory) The structure forces preservation of critical categories that free-form summaries silently drop. Cost: one LLM call (~20K–40K input + 2K–4K output) per compaction event.

Tier 3 — Full compaction with external backup (at 85% capacity): Summarize the entire conversation into a dense state document, clear the context, and restart with the summary as a "virtual file." Store the pre-compaction context in external storage for potential RAG retrieval. Will Larson's practical pattern stores the prior context window as a retrievable file, giving the agent a lifeline if the summary misses something.

RAG over conversation history is the most token-efficient approach (90–95% reduction) but weakest for maintaining sequential coherence. (Emergent Mind) It works best as a supplementary retrieval mechanism combined with a sliding window, not as a primary context management strategy. Embedding latency is minimal (~50ms per message) but requires vector DB infrastructure. (Mem0)

The critical failure mode across all compaction strategies is **artifact trail loss**—all methods scored poorly (2.19–2.45 out of 5) on tracking which files were created or modified. (factory) This is precisely why the user's workspace.md pattern is valuable: it provides a persistent record that survives compaction events. **Set compaction thresholds at 50–60% of the context window**, not 75–80%, as research shows model performance degrades well before the window fills.

Structured workspace formats resist bloat systematically

The "rewrite, don't append" instruction fights LLM tendencies but doesn't win structurally. LLMs are trained to be thorough and avoid losing information, creating a natural upward pressure on workspace size. The solution is **structural constraints with programmatic enforcement**.

The most effective workspace format uses **section headers with per-section token budgets**:

```
## Current State [max 150 tokens]
Phase: implementation | Active: src/handler.ts
Objective: implement retry logic for API calls

## Key Decisions [max 150 tokens]
[HIGH] Using exponential backoff (step 23)
[HIGH] Max 3 retries per request (step 31)

## Open Questions [max 100 tokens]
[MED] Whether to add circuit breaker pattern

## Completed [max 100 tokens]
✅ Auth module (step 15) ✅ Rate limiter (step 28)
```

Priority tags (**[HIGH]**/**[MED]**/**[LOW]**) enable automated pruning: HIGH items always survive compaction, MED items survive if within budget, LOW items are pruned first. **Step numbers** attached to entries enable temporal decay—entries older than 50 steps automatically lose detail, entries older than 100 steps get removed entirely.

Programmatic enforcement is essential. After each workspace rewrite, count tokens using tiktoken or the provider's tokenizer. If the count exceeds the budget (e.g., 500 tokens), trigger a second LLM call with a hard constraint: "Rewrite workspace.md to fit within 500 tokens, preserving all [HIGH] items." This costs one extra LLM call occasionally but prevents unbounded growth that would tax every subsequent call.

Separate volatile from stable state. "Current objective" changes every few steps; "architecture decisions" rarely change. Claude Code's approach—loading only the first 200 lines of MEMORY.md—reflects this principle. The MemGPT/Letta framework formalizes it further: a fixed-size "core memory" block for active state plus unlimited "archival memory" accessed through tool calls. (Letta)

Cost model: current vs. optimized architecture on Claude Sonnet

The following model uses Claude Sonnet 4 at \$3.00/MTok input, \$0.30/MTok cached read, \$3.75/MTok cache write (5-min TTL), (Claude API Docs) and \$15.00/MTok output. A 200-call job with 4 compaction cycles.

Current architecture (no caching, no optimization):

Component	Tokens/call	Total (200 calls)	Cost at \$3/MTok
System prompt	600	120,000	\$0.36
Phase prompt	300	60,000	\$0.18
Workspace.md	500 avg	100,000	\$0.30
instructions.md	2,000	400,000	\$1.20
Tool schemas	5,000	1,000,000	\$3.00
Conversation history	20,000 avg	4,000,000	\$12.00
Total input	28,400 avg	5,680,000	\$17.04
Output (~800/call)	800	160,000	\$2.40
Grand total			\$19.44

Optimized architecture (caching + tool compression + workspace budget + aggressive compaction):

Changes applied: (1) Prompt reordered for caching with 4 breakpoints, (2) tool schemas compressed from 5,000 → 3,500 tokens, (3) workspace.md budgeted to 400 tokens average, (4) compaction triggered at 50% context capacity to keep average history at ~12,000 tokens, (5) instructions.md kept in cached prefix.

Component	Tokens/call	Effective cost	Notes
Tool schemas (cached)	3,500	$3,500 \times \$0.30/\text{MTok} \times 195 \text{ reads} = \mathbf{\$0.21}$	5 write events at \$3.75: \$0.07
System prompt (cached)	600	Part of cached prefix	Included above
instructions.md (cached)	2,000	$2,000 \times \$0.30/\text{MTok} \times 195 = \mathbf{\$0.12}$	Cached with tools+system
Phase prompt (mostly cached)	300	$300 \times \$0.30 \times 194 + 300 \times \$3.75 \times 6 = \mathbf{\$0.02 + \$0.01}$	6 phase changes
Workspace.md (uncached)	400	$400 \times \$3.00/\text{MTok} \times 200 = \mathbf{\$0.24}$	Changes most calls
Conversation history (uncached)	12,000 avg	$12,000 \times \$3.00/\text{MTok} \times 200 = \mathbf{\$7.20}$	More aggressive compaction
Compaction LLM calls	~4 calls $\times$ 30K input	$120,000 \times \$3/\text{MTok} = \mathbf{\$0.36}$	Plus ~16K output: \$0.24
Total input cost			\$8.47
Output (~800/call)	800	$160,000 \times \$15/\text{MTok} = \mathbf{\$2.40}$	Unchanged
Grand total			\$10.87

Savings breakdown:

Optimization	Savings	% of total
Prompt caching (prefix at 0.1 $\times$)	\$3.91	20.1%
Earlier compaction (20K $\rightarrow$ 12K avg history)	\$4.80	24.7%
Tool compression (5K $\rightarrow$ 3.5K)	\$0.45	2.3%
Workspace budget (500 $\rightarrow$ 400)	\$0.06	0.3%
Compaction overhead (cost)	-\$0.60	-3.1%
Net savings	\$8.57	44.1%

The **biggest lever is conversation history management** (\$4.80 savings), followed by **prompt caching** (\$3.91). Tool compression and workspace budgets contribute incrementally. The compaction calls cost \$0.60 in overhead but save \$4.80 in reduced history size—an **8 $\times$ return**.

For higher-volume scenarios (500 calls/job or jobs running on Claude Opus at \$15/MTok input), the same optimizations scale proportionally. (MetaCTO) On Opus, the optimized architecture would save ~\$42 per job versus the unoptimized baseline.

Practical implementation roadmap

Week 1 — High impact, low effort: Reorder prompt components by stability and enable Anthropic prompt caching with 4 breakpoints. (Claude API Docs) (claude) Ensure deterministic tool definition ordering. This requires only prompt restructuring with no architectural changes. Expected savings: ~23% on input costs.

Week 2 — Medium impact, low effort: Compress tool descriptions to minimum viable length (remove optional parameter descriptions, shorten tool descriptions to disambiguation-only). (OpenAI) Add a hard token budget to workspace.md with programmatic enforcement. Expected additional savings: ~3–5%.

Week 3 — High impact, medium effort: Implement tiered context compaction. Start with observation masking at 50% capacity (zero LLM cost), (AWS) structured summarization at 70% capacity. This is the single highest-dollar optimization because it attacks the conversation history—the dominant cost component. Expected additional savings: ~15–25%.

Month 2 — Medium impact, high effort: Evaluate dynamic tool loading (Tool Search pattern) for phases where only a subset of tools is relevant. (Anthropic) Consider model routing—using Haiku (\$1/\$5) for simple file-read and classification steps, Sonnet for complex reasoning. (AI Free API) Model routing alone can reduce costs by 60–70% on eligible steps, (Zenvanriel) though it requires confidence-based routing logic.

The compounding effect matters most: a system that keeps conversation history short *and* caches the stable prefix *and* compresses tools operates at roughly 45–55% of the unoptimized cost. Over hundreds of jobs, the infrastructure investment in proper context engineering pays for itself many times over.